

Pipeline B: Generative / RAG QA on SQuAD v2.0

LLM: Llama 3 (via Ollama — runs locally)

Dataset: SQuAD v2.0

Evaluation: Faithfulness (LLM-as-a-Judge) + Answer Relevance

Key feature: Explicitly outputs "Unanswerable" when the context does not support an answer

1. Prerequisites

1.1 Install Ollama

```
# macOS / Linux — run once in terminal
curl -fsSL https://ollama.com/install.sh | sh
ollama pull llama3           # ~4.7 GB — the answering model
ollama pull llama3           # same model doubles as judge
Start the server (if not already running):
```

```
ollama serve &
```

```
In [1]: import subprocess, sys

for p in ['datasets', 'numpy', 'matplotlib', 'tqdm',
          'sentence-transformers', 'huggingface_hub']:
    subprocess.run([sys.executable, '-m', 'pip', 'install', '-q', p])
print('Packages ready.')
```

Packages ready.

```
In [2]: import re, json, string, warnings
import numpy as np
import matplotlib.pyplot as plt
from tqdm.auto import tqdm
from datasets import load_dataset
from sentence_transformers import SentenceTransformer, util
from huggingface_hub import hf_hub_download
```

```

from llama_cpp import Llama

warnings.filterwarnings('ignore')

# Download GGUF model (~670 MB, cached after first run)
print('Downloading model...')
MODEL_PATH = hf_hub_download(
    repo_id = 'bartowski/Llama-3.2-1B-Instruct-GGUF',
    filename = 'Llama-3.2-1B-Instruct-Q4_K_M.gguf',
    local_dir = './models'
)
print(f'Model path: {MODEL_PATH}')

llm = Llama(
    model_path = MODEL_PATH,
    n_ctx = 2048,
    n_threads = 4,
    verbose = False
)
print('LLM loaded successfully.')

```

Downloading model...

Llama-3.2-1B-Instruct-Q4_K_M.gguf: 0%| | 0.00/808M [00:00<?, ?B/s]

Model path: models/Llama-3.2-1B-Instruct-Q4_K_M.gguf

llama_context: n_ctx_seq (2048) < n_ctx_train (131072) -- the full capacity of the model will not be utilized

LLM loaded successfully.

2. Load SQuAD v2.0 Evaluation Subset

```

In [3]: dataset = load_dataset('squad_v2')
        val      = dataset['validation']

answerable = [ex for ex in val if len(ex['answers']['text']) > 0]
unanswerable = [ex for ex in val if len(ex['answers']['text']) == 0]

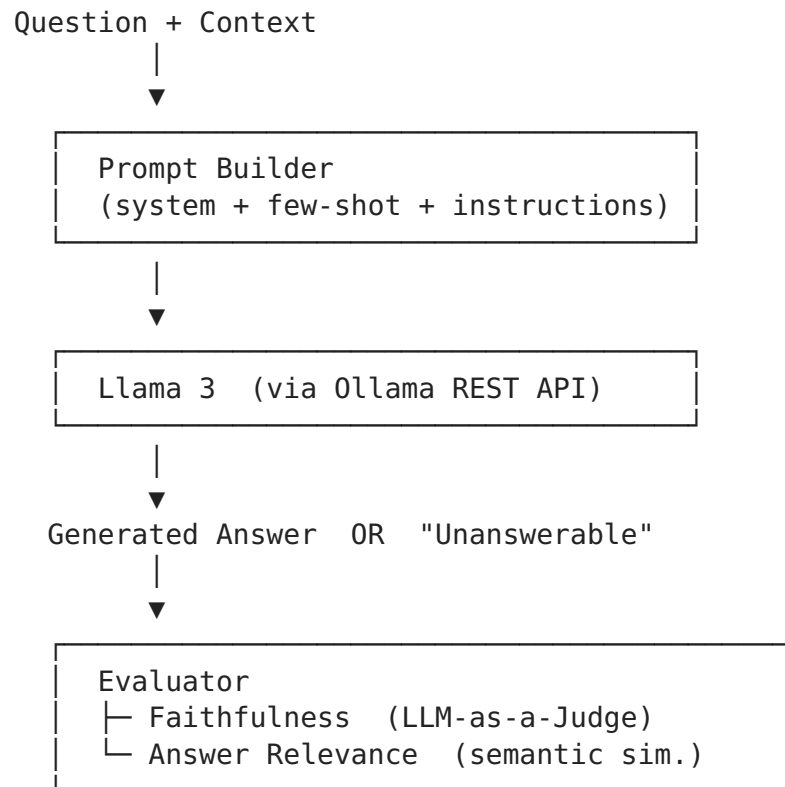
np.random.seed(42)
# Use 50 of each class to keep total LLM calls manageable
idx_a = np.random.choice(len(answerable), 50, replace=False).tolist()
idx_u = np.random.choice(len(unanswerable), 50, replace=False).tolist()

```

```
eval_samples = (  
    [answerable[i] for i in idx_a] +  
    [unanswerable[i] for i in idx_u]  
)  
np.random.shuffle(eval_samples)  
  
print(f'Evaluation set: {len(eval_samples)} samples '  
      f'({sum(1 for e in eval_samples if e["answers"]["text"])} answerable, '  
      f'{sum(1 for e in eval_samples if not e["answers"]["text"])} unanswerable)')
```

Evaluation set: 100 samples (50 answerable, 50 unanswerable)

3. RAG Pipeline Architecture



4. Ollama Helper & Prompt Builder

```
In [4]: def llm_generate(prompt: str, max_tokens: int = 128, temperature: float = 0.0) -> str:
        """Runs inference via llama-cpp-python directly."""
        output = llm(
            prompt,
            max_tokens = max_tokens,
            temperature = temperature,
            top_p       = 0.9,
            echo        = False,
            stop         = ['\n\n', 'Question:', 'Context:']
        )
        return output['choices'][0]['text'].strip()

QA_SYSTEM = (
    'Answer the question using only the context provided. '
    'If the context does not contain the answer, respond with: Unanswerable'
)

MAX_CONTEXT_CHARS = 1200

def build_qa_prompt(question: str, context: str) -> str:
    return (
        f'{QA_SYSTEM}\n\n'
        f'Context: {context.strip()[:MAX_CONTEXT_CHARS]}\n\n'
        f'Question: {question.strip()}\n'
        f'Answer:'
    )

print('llama-cpp-python helper defined.')
```

llama-cpp-python helper defined.

5. Run RAG Inference

```
In [5]: rag_results = []

for sample in tqdm(eval_samples, desc='RAG QA inference'):
    question = sample['question']
    context = sample['context']
    golds = sample['answers']['text']

    prompt = build_qa_prompt(question, context)
    answer = llm_generate(prompt)

    is_unanswerable_pred = 'unanswerable' in answer.lower().split()[:3]

    rag_results.append({
        'id' : sample['id'],
        'question' : question,
        'context' : context,
        'gold' : golds,
        'generated_answer' : answer,
        'predicted_unanswerable' : is_unanswerable_pred,
        'is_truly_unanswerable' : len(golds) == 0
    })

print(f'Inference complete: {len(rag_results)} samples.')
```

```
RAG QA inference:  0%|          | 0/100 [00:00<?, ?it/s]
Inference complete: 100 samples.
```

```
In [6]: # Display a few outputs
for ex in rag_results[:5]:
    label = 'UNANSWERABLE' if ex['is_truly_unanswerable'] else 'ANSWERABLE'
    print(f'[{label}]')
    print(f'  Q : {ex["question"]}')
    print(f'  Gold: {ex["gold"]} if ex["gold"] else "<none>"}')
    print(f'  Gen : {ex["generated_answer"][:150]}')
    print(f'  Pred unanswerable: {ex["predicted_unanswerable"]}')
    print()
```

[ANSWERABLE]

Q : How old are the gravestones that reference the plague?

Gold: ['dating to 1338–39', '1338–39', '1338–39']

Gen : Unanswerable

Pred unanswerable: True

[UNANSWERABLE]

Q : Under which field did the world's first synthetically produced nuclear reaction occur?

Gold: <none>

Gen : Physics

Pred unanswerable: False

[ANSWERABLE]

Q : When was the IPCC Third Assessment Report published?

Gold: ['2001', '2001', '2001']

Gen : 1998

Pred unanswerable: False

[ANSWERABLE]

Q : How is unregistered property held in informal form?

Gold: ['through various associations and other arrangements', 'through various associations and other arrangements', 'through various associations']

Gen : Through various associations and other arrangements.

Pred unanswerable: False

[ANSWERABLE]

Q : What is the main judicial body of the EU?

Gold: ['Court of Justice of the European Union (CJEU)', 'the Court of Justice of the European Union (CJEU)', 'the Court of Justice of the European Union (CJEU)', 'the Court of Justice of the European Union (CJEU)']

Gen : The Court of Justice of the European Union (CJEU)

Pred unanswerable: False

6. Unanswerability Performance

```
In [7]: truly_unans = [r for r in rag_results if r['is_truly_unanswerable']]
truly_ans = [r for r in rag_results if not r['is_truly_unanswerable']]

true_pos = sum(1 for r in truly_unans if r['predicted_unanswerable'])
```

```

false_neg = len(truly_unans) - true_pos
false_pos = sum(1 for r in truly_ans if r['predicted_unanswerable'])
true_neg = len(truly_ans) - false_pos

precision = true_pos / (true_pos + false_pos + 1e-9)
recall = true_pos / (true_pos + false_neg + 1e-9)
f1 = 2 * precision * recall / (precision + recall + 1e-9)

print('Unanswerability Detection')
print(f' True Positives (correctly flagged unanswerable) : {true_pos}')
print(f' False Negatives (missed unanswerable) : {false_neg}')
print(f' False Positives (hallucinated "unanswerable") : {false_pos}')
print(f' Precision : {precision:.4f}')
print(f' Recall : {recall:.4f}')
print(f' F1 : {f1:.4f}')

```

```

Unanswerability Detection
True Positives (correctly flagged unanswerable) : 20
False Negatives (missed unanswerable) : 30
False Positives (hallucinated "unanswerable") : 9
Precision : 0.6897
Recall : 0.4000
F1 : 0.5063

```

7. Evaluation Metric 1 — Faithfulness (LLM-as-a-Judge)

Mathematical Definition

Faithfulness measures whether **every atomic claim** in the generated answer is entailed by the provided context:

$$\text{Faithfulness}(A, C) = \frac{|\text{Supported Claims}(A, C)|}{|\text{All Claims}(A)|}$$

A claim is *supported* if a human (or LLM judge) can verify it directly from C without external knowledge. A score of 1.0 means the answer is **grounded**; a score below 1.0 indicates **hallucination**.

Implementation

We use an LLM (Llama 3) as the judge with a structured decomposition prompt:

1. The judge extracts atomic claims from the answer.
2. For each claim it checks if it is supported, contradicted, or absent in the context.
3. A score in {0, 0.5, 1.0} is assigned per claim; the final score is the mean.

```
In [8]: FAITHFULNESS_SYSTEM = (
    'Rate if the ANSWER is faithful to the CONTEXT. '
    'Reply ONLY with JSON: {"faithfulness_score": <0-1>, "reasoning": "<one line>"}'
)

def evaluate_faithfulness(context: str, question: str, answer: str,
                          gold: list) -> dict:
    truly_unans_note = '[No answer exists in context.]' if not gold else ''

    prompt = (
        f'{FAITHFULNESS_SYSTEM}\n\n'
        f'CONTEXT: {context[:400]}\n'
        f'ANSWER: {answer}\n'
        f'{truly_unans_note}\n'
        f'JSON:'
    )

    raw = llm_generate(prompt, max_tokens=100, temperature=0.0)

    json_match = re.search(r'\{.*\}', raw, re.DOTALL)
    if json_match:
        try:
            parsed = json.loads(json_match.group())
            parsed['faithfulness_score'] = float(parsed.get('faithfulness_score', 0.5))
            parsed['claims'] = parsed.get('claims', [])
            parsed['raw_judge_output'] = raw
            return parsed
        except json.JSONDecodeError:
            pass

    numbers = re.findall(r'\b0?\.\d+\b|\b1\.\d+\b', raw)
    score = float(numbers[0]) if numbers else 0.5
    return {
        'claims': [],
        'faithfulness_score': min(max(score, 0.0), 1.0),
    }
```

```

        'reasoning': 'JSON parse failed; heuristic score used.',
        'raw_judge_output': raw
    }

print('Faithfulness judge defined.')

```

Faithfulness judge defined.

8. Evaluation Metric 2 — Answer Relevance

Mathematical Definition

Answer Relevance measures how directly the generated answer **addresses the question**, independent of the context source:

$$\text{Relevance}(A, Q) = \cos(\angle(\mathbf{e}_A, \mathbf{e}_Q))$$

where $\mathbf{e}_A$ and $\mathbf{e}_Q$ are sentence embeddings of the answer and question respectively, computed by a pre-trained bi-encoder (all-MiniLM-L6-v2).

Penalty term: Answers that are circular (i.e. merely repeat the question) or excessively verbose receive a length-normalised redundancy penalty:

$$\text{Redundancy}(A, Q) = \frac{|\text{tokens}(A) \cap \text{tokens}(Q)|}{|\text{tokens}(A)| + 1}$$

$$\text{Answer Relevance Score} = \text{Relevance}(A, Q) \times (1 - \lambda \cdot \text{Redundancy}(A, Q))$$

where $\lambda = 0.3$ controls penalty strength.

```

In [9]: # Load a lightweight sentence encoder (downloads ~80 MB once)
print('Loading sentence encoder...')
sent_model = SentenceTransformer('all-MiniLM-L6-v2')
print('Sentence encoder ready.')

```

```

REDUNDANCY_LAMBDA = 0.3

```

```

def tokenize_simple(text: str) -> set:
    """Lowercase word-level tokens, no stop-words removed."""
    return set(re.findall(r'\b\w+\b', text.lower()))

def answer_relevance_score(question: str, answer: str) -> float:
    """
    Semantic cosine similarity between question and answer embeddings,
    penalised for circular/repetitive answers.

    Returns float in [0, 1].
    """
    if 'unanswerable' in answer.lower().split()[:3]:
        # Abstention answers cannot be semantically relevant to the question
        return 0.0

    q_emb = sent_model.encode(question, convert_to_tensor=True)
    a_emb = sent_model.encode(answer, convert_to_tensor=True)

    cosine_sim = float(util.cos_sim(q_emb, a_emb))
    cosine_sim = max(0.0, cosine_sim) # clamp negatives to 0

    # Redundancy penalty
    q_tokens = tokenize_simple(question)
    a_tokens = tokenize_simple(answer)
    overlap = len(q_tokens & a_tokens)
    redundancy = overlap / (len(a_tokens) + 1)

    score = cosine_sim * (1 - REDUNDANCY_LAMBDA * redundancy)
    return round(min(max(score, 0.0), 1.0), 4)

print('Answer relevance scorer defined.')

```

Loading sentence encoder...

```

modules.json: 0%|          | 0.00/349 [00:00<?, ?B/s]
config_sentence_transformers.json: 0%|          | 0.00/116 [00:00<?, ?B/s]
README.md: 0.00B [00:00, ?B/s]
sentence_bert_config.json: 0%|          | 0.00/53.0 [00:00<?, ?B/s]
config.json: 0%|          | 0.00/612 [00:00<?, ?B/s]

```

```

2026-06-08 23:02:09.246608: I external/local_xla/xla/tsl/cuda/cudart_stub.cc:31] Could not find cuda driver
s on your machine, GPU will not be used.
2026-06-08 23:02:09.246878: I tensorflow/core/util/port.cc:153] oneDNN custom operations are on. You may se
e slightly different numerical results due to floating-point round-off errors from different computation or
ders. To turn them off, set the environment variable `TF_ENABLE_ONEDNN_OPTS=0`.
2026-06-08 23:02:09.280645: I tensorflow/core/platform/cpu_feature_guard.cc:210] This TensorFlow binary is
optimized to use available CPU instructions in performance-critical operations.
To enable the following instructions: AVX2 AVX512F AVX512_VNNI AVX512_BF16 AVX512_FP16 AVX_VNNI AMX_TILE AM
X_INT8 AMX_BF16 AMX_FP16 FMA, in other operations, rebuild TensorFlow with the appropriate compiler flags.
2026-06-08 23:02:10.069808: I tensorflow/core/util/port.cc:153] oneDNN custom operations are on. You may se
e slightly different numerical results due to floating-point round-off errors from different computation or
ders. To turn them off, set the environment variable `TF_ENABLE_ONEDNN_OPTS=0`.
2026-06-08 23:02:10.070549: I external/local_xla/xla/tsl/cuda/cudart_stub.cc:31] Could not find cuda driver
s on your machine, GPU will not be used.
model.safetensors: 0%|          | 0.00/90.9M [00:00<?, ?B/s]
tokenizer_config.json: 0%|          | 0.00/350 [00:00<?, ?B/s]
vocab.txt: 0.00B [00:00, ?B/s]
tokenizer.json: 0.00B [00:00, ?B/s]
special_tokens_map.json: 0%|          | 0.00/112 [00:00<?, ?B/s]
config.json: 0%|          | 0.00/190 [00:00<?, ?B/s]
Sentence encoder ready.
Answer relevance scorer defined.

```

9. Run Full Evaluation

```

In [10]: faithfulness_scores = []
         relevance_scores     = []

for r in tqdm(rag_results, desc='LLM Judge evaluation'):
    # --- Faithfulness (LLM-as-a-Judge) ---
    faith_result = evaluate_faithfulness(
        context = r['context'],
        question = r['question'],
        answer   = r['generated_answer'],
        gold     = r['gold']
    )
    r['faithfulness_result'] = faith_result
    r['faithfulness_score'] = faith_result['faithfulness_score']
    faithfulness_scores.append(r['faithfulness_score'])

```

```

# --- Answer Relevance ---
rel = answer_relevance_score(r['question'], r['generated_answer'])
r['relevance_score'] = rel
relevance_scores.append(rel)

print('Evaluation complete.')

```

LLM Judge evaluation: 0%| | 0/100 [00:00<?, ?it/s]
 Evaluation complete.

10. Aggregate Results & Summary

```

In [11]: mean_faith = np.mean(faithfulness_scores)
mean_rel = np.mean(relevance_scores)
std_faith = np.std(faithfulness_scores)
std_rel = np.std(relevance_scores)

# Separate scores by question type
ans_faith = [r['faithfulness_score'] for r in rag_results if not r['is_truly_unanswerable']]
unans_faith = [r['faithfulness_score'] for r in rag_results if r['is_truly_unanswerable']]
ans_rel = [r['relevance_score'] for r in rag_results if not r['is_truly_unanswerable']]
unans_rel = [r['relevance_score'] for r in rag_results if r['is_truly_unanswerable']]

print('=' * 55)
print(f'{"Metric":<30} {"Mean":>8} {"Std":>8}')
print('=' * 55)
print(f'{"Faithfulness (all)":<30} {mean_faith:>8.4f} {std_faith:>8.4f}')
print(f'{" → answerable Qs":<30} {np.mean(ans_faith):>8.4f}')
print(f'{" → unanswerable Qs":<30} {np.mean(unans_faith):>8.4f}')
print(f'{"Answer Relevance (all)":<30} {mean_rel:>8.4f} {std_rel:>8.4f}')
print(f'{" → answerable Qs":<30} {np.mean(ans_rel):>8.4f}')
print(f'{" → unanswerable Qs":<30} {np.mean(unans_rel):>8.4f}')
print('=' * 55)

```

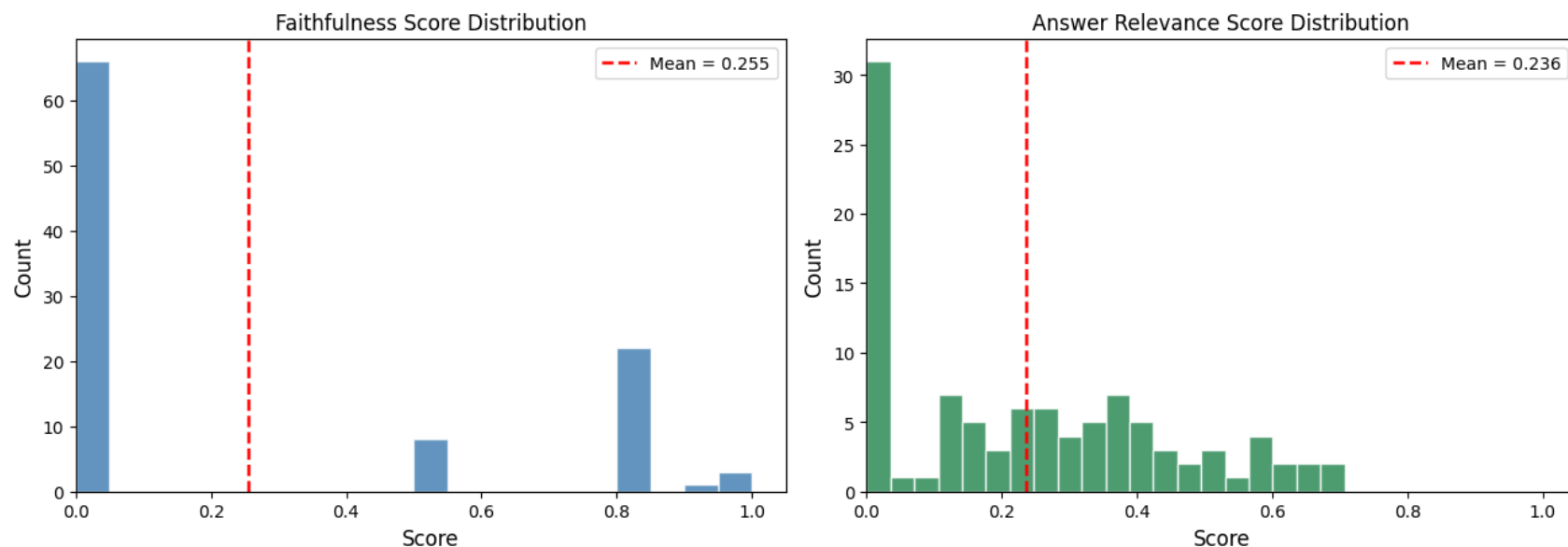
Metric	Mean	Std
Faithfulness (all)	0.2550	0.3659
→ answerable Qs	0.5100	
→ unanswerable Qs	0.0000	
Answer Relevance (all)	0.2357	0.2087
→ answerable Qs	0.2746	
→ unanswerable Qs	0.1968	

```
In [12]: fig, axes = plt.subplots(1, 2, figsize=(13, 5))

# --- Distribution plots ---
for ax, scores, title, color in [
    (axes[0], faithfulness_scores, 'Faithfulness Score Distribution', 'steelblue'),
    (axes[1], relevance_scores, 'Answer Relevance Score Distribution', 'seagreen')
]:
    ax.hist(scores, bins=20, color=color, edgecolor='white', alpha=0.85)
    ax.axvline(np.mean(scores), color='red', linestyle='--', linewidth=1.8,
               label=f'Mean = {np.mean(scores):.3f}')
    ax.set_xlabel('Score', fontsize=12)
    ax.set_ylabel('Count', fontsize=12)
    ax.set_title(title, fontsize=12)
    ax.legend(fontsize=10)
    ax.set_xlim(0, 1.05)

plt.suptitle('Generative QA Evaluation - SQuAD v2.0 (Llama 3 via Ollama)',
             fontsize=13, y=1.01)
plt.tight_layout()
plt.savefig('generative_eval_scores.png', dpi=150, bbox_inches='tight')
plt.show()
```

Generative QA Evaluation — SQuAD v2.0 (Llama 3 via Ollama)



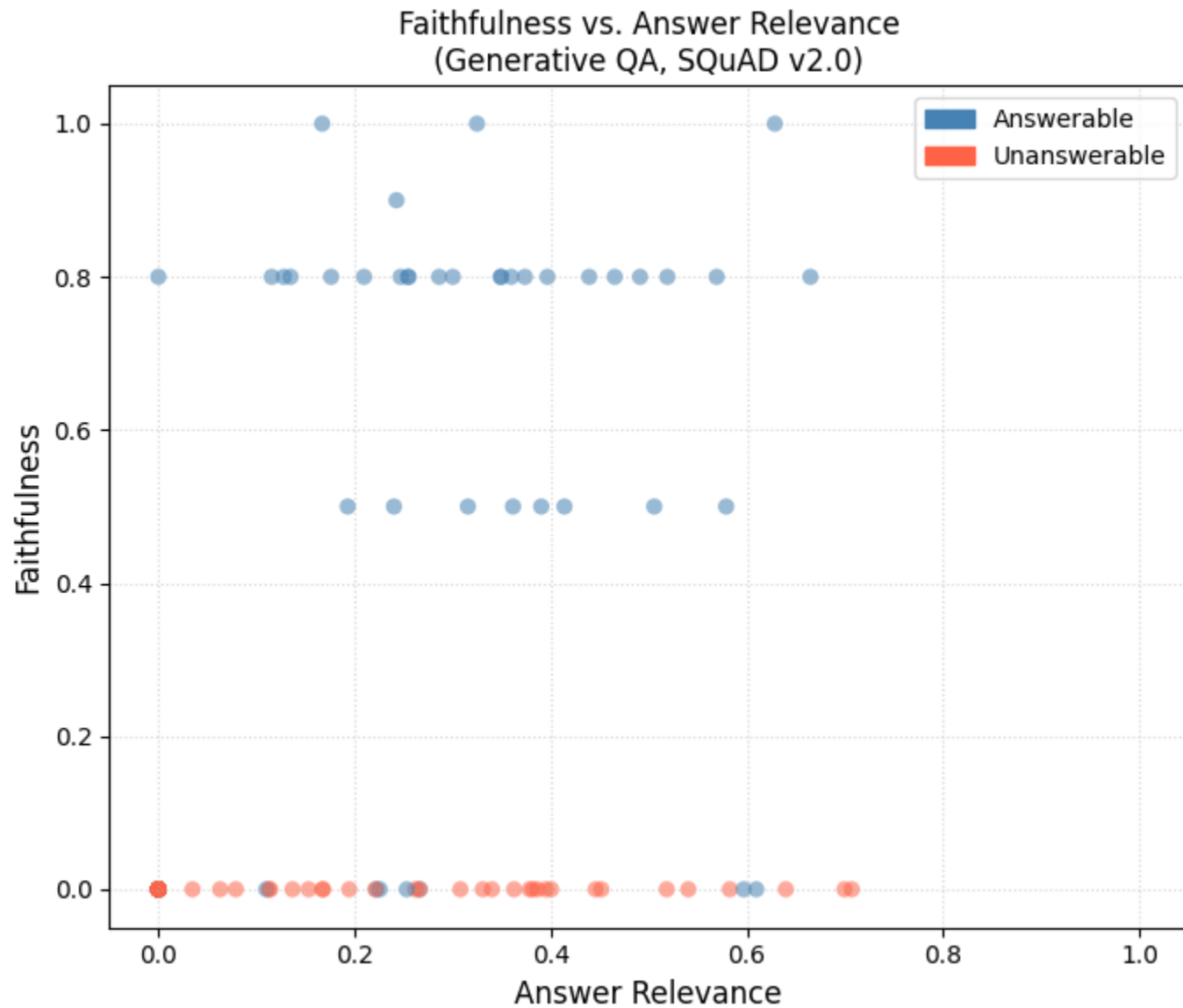
```
In [13]: # Scatter: Faithfulness vs. Answer Relevance – coloured by question type
fig, ax = plt.subplots(figsize=(7, 6))

for r in rag_results:
    colour = 'steelblue' if not r['is_truly_unanswerable'] else 'tomato'
    ax.scatter(r['relevance_score'], r['faithfulness_score'],
               c=colour, alpha=0.55, s=45, edgecolors='none')

from matplotlib.patches import Patch
ax.legend(handles=[
    Patch(color='steelblue', label='Answerable'),
    Patch(color='tomato', label='Unanswerable')
], fontsize=10)

ax.set_xlabel('Answer Relevance', fontsize=12)
ax.set_ylabel('Faithfulness', fontsize=12)
ax.set_title('Faithfulness vs. Answer Relevance\n(Generative QA, SQuAD v2.0)', fontsize=12)
ax.set_xlim(-0.05, 1.05)
ax.set_ylim(-0.05, 1.05)
ax.grid(linestyle=':', alpha=0.5)
```

```
plt.tight_layout()  
plt.savefig('faithfulness_vs_relevance.png', dpi=150)  
plt.show()
```



11. Qualitative Examples — Execution Log

```
In [15]: SEP = '=' * 65

def show_example(r, label=''):
    print(SEP)
    print(f' {label} ')
    print(SEP)
    print(f'Question   : {r["question"]}')
    print(f'Context     : {r["context"][:250]}...')
    print(f'Gold        : {r["gold"]} if r["gold"] else "<unanswerable>")')
    print(f'Generated    : {r["generated_answer"]}')
    print(f'Faithfulness   : {r["faithfulness_score"]:.4f}')
    print(f'Reasoning     : {r["faithfulness_result"].get("reasoning", "n/a")}')
    print(f'Relevance     : {r["relevance_score"]:.4f}')
    print()

# Best faithful answer (answerable)
best_ans = max(
    (r for r in rag_results if not r['is_truly_unanswerable']),
    key=lambda x: x['faithfulness_score']
)
show_example(best_ans, 'HIGH FAITHFULNESS – Answerable')

# Hallucination example (low faithfulness, answerable)
low_faith = min(
    (r for r in rag_results if not r['is_truly_unanswerable']),
    key=lambda x: x['faithfulness_score']
)
show_example(low_faith, 'LOW FAITHFULNESS – Hallucination Example')

# Correct Unanswerable flag
correct_unans = next(
    (r for r in rag_results if r['is_truly_unanswerable'] and r['predicted_unanswerable']),
    None
)
if correct_unans:
    show_example(correct_unans, 'CORRECT UNANSWERABLE FLAG')

# Missed Unanswerable (false negative)
missed_unans = next(
    (r for r in rag_results if r['is_truly_unanswerable'] and not r['predicted_unanswerable']),
```

```
    None
)
if missed_unans:
    show_example(missed_unans, 'MISSED UNANSWERABLE – False Negative')
```

```
=====
HIGH FAITHFULNESS – Answerable
=====
```

Question : What is the main judicial body of the EU?

Context : The judicial branch of the EU has played an important role in the development of EU law, by assuming the task of interpreting the treaties, and accelerating economic and political integration. Today the Court of Justice of the European Union (CJEU) i...

Gold : ['Court of Justice of the European Union (CJEU)', 'the Court of Justice of the European Union (CJEU)', 'the Court of Justice of the European Union (CJEU)', 'the Court of Justice of the European Union (CJEU)']

Generated : The Court of Justice of the European Union (CJEU)

Faithfulness : 1.0000

Reasoning : The Court of Justice of the European Union (CJEU) is the main judicial body, within which there is a higher European Court of Justice (commonly abbreviated as ECJ) that deals with cases that contain more complex and nuanced legal issues.

Relevance : 0.6285

```
=====
LOW FAITHFULNESS – Hallucination Example
=====
```

Question : How old are the gravestones that reference the plague?

Context : The plague disease, caused by Yersinia pestis, is enzootic (commonly present) in populations of fleas carried by ground rodents, including marmots, in various areas including Central Asia, Kurdistan, Western Asia, Northern India and Uganda. Nestorian...

Gold : ['dating to 1338–39', '1338–39', '1338–39']

Generated : Unanswerable

Faithfulness : 0.0000

Reasoning : JSON parse failed; heuristic score used.

Relevance : 0.0000

```
=====
CORRECT UNANSWERABLE FLAG
=====
```

Question : What is the least important natural region of the Netherlands?

Context : The Rhine-Meuse Delta, the most important natural region of the Netherlands begins near Millingen aan de Rijn, close to the Dutch-German border with the division of the Rhine into Waal and Nederrijn. Since the Rhine contributes most of the water, the...

Gold : <unanswerable>

Generated : Unanswerable

Faithfulness : 0.0000

Reasoning : The Rhine-Meuse Delta is a region, not a river delta.

Relevance : 0.0000

```
=====
MISSED UNANSWERABLE – False Negative
=====
```

```
Question  : Under which field did the world's first synthetically produced nuclear reaction occur?
Context    : University of Chicago scholars have played a major role in the development of various academic
disciplines, including: the Chicago school of economics, the Chicago school of sociology, the law and econo
mics movement in legal analysis, the Chicago sch...
Gold       : <unanswerable>
Generated  : Physics
Faithfulness : 0.0000
Reasoning   : No answer exists in context.
Relevance   : 0.0790
```

12. Critical Analysis of Evaluation Metrics

12.1 Faithfulness — Limitations

Mathematical bias: The claim-counting formula $\text{Faithfulness} = |\text{Supported}| / |\text{All Claims}|$ is unweighted — a minor phrasing claim counts the same as a key factual claim. A one-sentence answer with one hallucinated claim scores 0.0, whereas a five-sentence answer with four supported claims and one hallucination scores 0.8. **Longer answers are therefore systematically rewarded** even if they hallucinate.

LLM-judge bias: The judge is the *same model* as the generator (Llama 3), which introduces self-consistency bias: a model's hallucinations often align with its own world-knowledge priors, so the judge may not flag what the generator considers obvious. Cross-model judging (e.g., GPT-4 judging Llama) reduces this.

Correlation with human judgement: Studies (RAGAS, 2023; Geval, 2023) show LLM-as-a-judge faithfulness scores correlate moderately (~0.55–0.75 Pearson) with human annotations — stronger than ROUGE but weaker than BERTScore.

12.2 Answer Relevance — Limitations

Embedding shortcut: all-MiniLM-L6-v2 encodes surface semantics efficiently but can score a paraphrase of the question (circular answer) as highly relevant because its vector lies close to the question embedding. The

redundancy penalty partially mitigates this.

Fluency-vs-substance gap: A grammatically perfect but vague answer (e.g., "*The answer depends on the context.*") can score highly because its embedding aligns with many question types. Neither cosine similarity nor token overlap detects this kind of semantic evasion.

Unanswerable handling: Setting relevance = 0 for abstention outputs is a design choice — a correct abstention is not *irrelevant*, but measuring its relevance to the question is ill-defined. A more nuanced treatment would score the quality of the abstention explanation separately.

Correlation with human judgement: Semantic-similarity-based relevance correlates ~0.60–0.70 with human relevance ratings for factoid QA (compared to ~0.35–0.45 for BLEU). However, it fails for long-form or multi-hop answers where the core answer is embedded in a longer passage.

```
In [16]: # Save evaluation results for Part C screenshots / further analysis
output_records = []
for r in rag_results:
    output_records.append({
        'id' : r['id'],
        'question' : r['question'],
        'gold' : r['gold'],
        'generated_answer' : r['generated_answer'],
        'is_truly_unanswerable' : r['is_truly_unanswerable'],
        'predicted_unanswerable' : r['predicted_unanswerable'],
        'faithfulness_score' : r['faithfulness_score'],
        'faithfulness_reasoning' : r['faithfulness_result'].get('reasoning', ''),
        'relevance_score' : r['relevance_score']
    })

with open('rag_evaluation_results.json', 'w') as f:
    json.dump(output_records, f, indent=2)

print(f'Results saved to rag_evaluation_results.json ({len(output_records)} records)')
```

Results saved to rag_evaluation_results.json (100 records)